

1. LINDO — Introduction

(Stand in front, screen closed or on a title slide)

"Good morning/afternoon everyone. We are presenting our Assignment Two for Applications Development 3B. Our project is called **Vision Analyzer**. It's a web application that uses Microsoft Cognitive Services — specifically the Computer Vision AI. The concept is simple: you upload a photo, and within seconds, the AI tells you exactly what's in that picture using plain English."

[Cue: Akhona takes over]

2. AKHONA — What the app does

(Screen shows the homepage with "Point. Upload. Understand.")

"The idea is straightforward. Most of us take many photos but never organize or describe them. Our app solves this using AI. You upload any image, and the system reads objects, scenes, and context automatically. It then gives you a written description — like having a smart assistant analyze your photo instantly."

[Cue: Azania takes over]

3. AZANIA — The architecture / Azure services

(Homepage or simple architecture slide)

"We used three main Azure cloud services for this project. **First**, Azure Blob Storage for storing images. **Second**, Azure Computer Vision — the actual AI that processes the image and generates the description. **Third**, Azure Cache for Redis, which stores previous results so we don't call the AI twice for the same image."

[Cue: Sabza takes over]

4. SABZA — Blob Storage

(Show Azure Portal Storage account / containers briefly, or stay on app)

"To store our images, we used **Azure Blob Storage**, as specified in the assignment guide. This is a cloud-based storage service where every photo uploaded through our app is stored securely. Each image receives a permanent link, and that link is used to display the photo on our results page. Storing images this way ensures they are always accessible and scalable for large usage."

[Cue: hand laptop/control to presenter doing live demo — Me]

5. ME — Live Demo Part 1: Upload

(Actually do this live)

"This is our homepage. It was built using [ASP.NET Core MVC](#) — the framework we're using for this project. I will now upload a photo to demonstrate the application."

(Select an image, click the Analyze button, wait for result page to load)

[Cue: Anele takes over as soon as the Result page appears]

6. ANELE — Live Demo Part 2: The AI Result

(Screen now shows Result page with image, description, and tags)

"And here is the result in just a few seconds. Azure Computer Vision has read this image and provided a **description**, a **confidence score** — showing how sure the AI is about its analysis — and a list of **detected tags**. Below that, you see our full written analysis. This takes the AI's raw output and turns it into a proper, readable paragraph, almost like a mini report."

[Cue: Mr Right takes over]

7. MR RIGHT — How the data flows through the system

(Stay on the Result page that's already showing)

"To explain briefly how the data moves through our system: when a user uploads an image, the app first sends it to **Azure Blob Storage** to save the file. Then it calls the **Computer Vision API** to analyze the photo. Finally, the results — description, tags, and confidence — are displayed on this screen. The entire flow is managed by our [ASP.NET Core MVC](#) controller, which coordinates all these services behind the scenes."

[Cue: Fanele takes over]

8. FANELE — Code architecture / OOP practices

(Show Visual Studio Solution Explorer with Services folder)

"We followed proper **Object-Oriented Programming** practices. Instead of putting all the logic into one big Controller class, we split our code into separate services: one for Blob Storage operations, one for Computer Vision API calls, and one for Redis Caching. Each service has its own interface. This is called **Dependency Injection**, and it makes our code clean, testable, reusable, and easier to maintain."

[Cue: Mthobisi takes over]

9. MTHOBISI — Challenges faced

(Stay on app or portal)

"Like any cloud project, we faced some real challenges. We had to fix connection string formatting issues, enable public access settings on our storage account, and handle cases where Redis might be temporarily unavailable. We built our app so that even if the cache fails, the core feature — image analysis — still works perfectly without crashing. This is called **defensive programming**, and it ensures our application remains reliable."

[Cue: Dunge takes over — closing]

10. DUNGE — Conclusion

(Back on homepage or a thank-you slide)

"To conclude — Vision Analyzer demonstrates a real, working cloud application using three Azure services together: **Blob Storage for file storage**, **Computer Vision for AI analysis**, and **Redis Cache for performance** — all wrapped in a clean, professional interface built with [ASP.NET](#) Core MVC. This is a solid example of how modern businesses build scalable, intelligent cloud applications on Microsoft Azure. Thank you — we're happy to take any questions."